

Here is a clean action plan using the title:

Learning When Factors Work: Regime-Based Factor Rotation with Bandit Algorithms

1. Summary and idea

We build a trading/allocation algorithm that decides once per month how to allocate across factor strategies.

The core hypothesis:

Factor premia are not constant over time. A learning algorithm can adapt to changing market regimes and rotate between factor exposures better than a static factor portfolio.

This is a good WUTIS project because it is not just "let's throw RL at trading". It is a clearly defined sequential decision problem:

- At time t , we observe the market regime.
- The model chooses an allocation rule.
- The portfolio earns next month's return.
- The model receives a reward based on return, risk, and turnover.
- Over time, the model learns which allocation rules work in which regimes.

This fits the RL framing from the book: finance problems can be cast as sequential decision problems under uncertainty, and dynamic asset allocation means adjusting capital allocation over time as circumstances change. The book also explicitly treats multi-armed bandits as the basic exploration-vs-exploitation framework and notes that these methods extend naturally to contextual bandits and RL control.

The project should not try to predict individual stock returns. That is too noisy. We work at the factor level.

2. Data availability

Main dataset:

Kenneth French Data Library

Use monthly factor returns:

- Market excess return: MKT - RF
- Size: SMB

- Value: HML
- Profitability: RMW
- Investment: CMA
- Momentum: MOM, from the momentum factor dataset
- Risk-free rate: RF

The Kenneth French Data Library provides historical U.S. monthly factor data, including the Fama/French 3-factor and 5-factor datasets, with monthly history going back to 1926 for the classic factors.

Optional additional data:

- VIX from Yahoo Finance
- SPY / S&P; 500 ETF prices from Yahoo Finance
- 10Y Treasury yield
- inflation or recession dummy

But for the first version, do not rely on too many external datasets. The clean version should work with Fama-French data alone.

Suggested dataset structure:

```
| Date | MKT_RF | SMB | HML | RMW | CMA | MOM | RF | regime features |
| --- | ---- | --- | ---- | ---- | ---- | ---- | ---- | --- |
```

Train/test split:

- Training: 1965-2005
- Validation: 2006-2014
- Test: 2015-latest available

Alternative:

- Expanding-window walk-forward backtest from 1980 onward.

The walk-forward version is better, but also more work.

3. Model

Decision frequency

Monthly.

At the end of each month t , the model observes historical information up to t , chooses an allocation for month $t+1$, and then receives the realized reward after month $t+1$.

No look-ahead. This is critical.

State / context variables

Use only variables known at decision time.

Recommended state vector:

```
s_t = [  
  R^MKT_{t-1:t-12},  
  sigma^MKT_{t-12:t},  
  DD^MKT_t,  
  MOM^factor_{t-12:t},  
  VOL^factor_{t-12:t},  
  Spread^best-worst_t  
]
```

In plain English:

- 12-month market return
- 3-month market return
- 12-month market volatility
- market drawdown from previous high
- previous 12-month return of each factor
- previous 12-month volatility of each factor
- difference between best and worst recent factor performance

Keep the state small. Too many features will make the project look more sophisticated but actually weaker.

Action space: complex allocation rules

Instead of choosing one factor directly, the model chooses between allocation rules.

That is much better.

Actions:

1. Equal-Weight Factors

Allocate equally to all factor portfolios.

2. Momentum Tilt

Overweight factors with the best past 12-month performance.

3. Defensive Tilt

Overweight low-volatility / profitability / investment-style factors and reduce market exposure.

4. Value-Reversal Tilt

Overweight factors that underperformed recently but have positive long-term average premia.

5. Risk-Parity Factor Allocation

Allocate inversely proportional to recent volatility.

6. Cash / Risk-Off Allocation

Move partially or fully into risk-free rate when regime looks bad.

7. Market Benchmark Allocation

Mostly market factor exposure.

Example action output:

$$w_t(a) = (w_{MKT}, w_{SMB}, w_{HML}, w_{RMW}, w_{CMA}, w_{MOM}, w_{RF})$$

Add constraints:

$$\sum_i w_{i,t} = 1$$

$$0 \leq w_{i,t} \leq 0.5$$

No shorting in the base version. Shorting can be added as a robustness test, but it makes interpretation harder.

Reward function: complex version

Use a risk- and cost-adjusted reward:

$$\text{Reward}_{t+1} = R_{p,t+1} - \lambda * \text{Turnover}_t - \gamma * \max(0, -R_{p,t+1})^2 - \eta * \text{DD}_{t+1}$$

Where:

- $R_{p,t+1}$ = next month portfolio return
- $\text{Turnover}_t = \sum_i |w_{i,t} - w_{i,t-1}|$
- $\max(0, -R_{p,t+1})^2$ penalizes downside returns
- DD_{t+1} penalizes drawdown
- λ, γ, η are penalty parameters

Interpretation:

The model should not just maximize raw return. It should prefer smoother returns, lower drawdown, and lower trading costs.

Start with parameter values like:

$\lambda = 0.001$

$\gamma = 2$

$\eta = 0.05$

Then run sensitivity checks.

Do not pretend these parameters are "true". They are preference parameters.

4. Backtesting, model types, benchmarks

Backtesting design

Use a strict walk-forward setup.

At each month:

1. Use only data available up to month t .
2. Compute context features.
3. Let the model select allocation rule a_t .
4. Convert selected rule into portfolio weights w_t .
5. Hold portfolio during month $t+1$.
6. Observe realized return.
7. Compute reward.
8. Update model.

This makes the project defensible.

Model types to implement

Minimum set:

Model 1: Static benchmark

Equal-weight factor portfolio.

This is the baseline to beat.

Model 2: Rolling Sharpe selector

Choose the allocation rule with the best rolling 36-month Sharpe.

This is a non-RL adaptive benchmark.

Model 3: epsilon-greedy bandit

Simple exploration-exploitation model.

Useful because it is easy to explain.

Model 4: UCB bandit

Chooses actions based on estimated reward plus uncertainty bonus.

This is better for presentation because the intuition is clean:

Try actions that either worked well or are still uncertain.

Model 5: Thompson Sampling

Bayesian-style probability matching.

Good final model because it sounds sophisticated but is still implementable.

Model 6: Contextual bandit / LinUCB

Main model.

It uses the current regime features to decide which allocation rule is best.

This is the "real" project model.

The book explicitly lists UCB, Bayesian UCB, Thompson Sampling, gradient bandits, and contextual extensions as part of the exploration-exploitation toolkit.

Benchmarks

Compare against:

1. Market factor / market benchmark
2. Static equal-weight factor portfolio
3. 60/40 proxy, optional
4. Best single factor in hindsight

5. Rolling best factor
6. Rolling Sharpe allocation
7. Random allocation rule, as sanity check

The "best single factor in hindsight" is not a realistic strategy, but it is useful as an upper reference.

Evaluation metrics

Use:

- annualized return
- annualized volatility
- Sharpe ratio
- Sortino ratio
- max drawdown
- Calmar ratio
- turnover
- transaction-cost-adjusted return
- hit rate versus benchmark
- performance in crisis periods

Important crisis windows:

- 2000-2002
- 2008-2009
- 2020
- 2022

The jury will care less about tiny Sharpe improvements and more about whether the model behaves plausibly in different regimes.

5. Interpretation

This is where the project wins or dies.

Do not just show "our model made more money". That is weak.

You need to explain why it allocated the way it did.

Key interpretation charts:

1. Allocation over time

Show stacked area chart of selected factor exposures.

Question answered:

When does the model prefer momentum, value, defensive factors, or cash?

2. Regime map

Group months into regimes:

- bull market / low volatility
- bear market / high volatility
- recovery period
- sideways / choppy market

Then show which allocation rules were selected in each regime.

3. Reward decomposition

Show how much return was lost to:

- turnover penalty
- downside penalty
- drawdown penalty

This makes the complex reward transparent.

4. Transaction cost sensitivity

Run the strategy under:

- 0 bps
- 5 bps
- 10 bps
- 25 bps

If the strategy only works at zero cost, say it clearly. Better to be honest than to oversell.

5. Robustness

Run:

- different train/test splits
- different rolling windows

- different penalty parameters
- with and without cash action
- with and without momentum factor

This prevents the project from looking like curve-fitting.

Final deliverable structure

Slides

1. Motivation: factor premia are time-varying
2. Research question
3. Data and factor universe
4. Sequential decision problem
5. Allocation rules
6. Reward function
7. Bandit models
8. Backtesting design
9. Results
10. Interpretation by regime
11. Robustness checks
12. Limitations
13. Conclusion

Code modules

Recommended Python structure:

```
project/  
  data/  
    raw/  
    processed/  
  src/  
    load_data.py  
    features.py  
    allocation_rules.py  
    rewards.py  
    bandits.py
```

```
backtest.py
metrics.py
plots.py
notebooks/
  01_data_check.ipynb
  02_baselines.ipynb
  03_bandit_models.ipynb
  04_results.ipynb
report/
  slides.pptx
```

3-week execution plan

Week 1: Data, baseline, framework

Goal: have a working backtest before touching fancy models.

Tasks:

- Download Fama-French factor data.
- Build monthly return table.
- Create factor allocation rules.
- Implement backtester.
- Implement static benchmarks.
- Produce first cumulative return and drawdown plots.

End of week 1 deliverable:

Equal-weight, market benchmark, and rule-based portfolios are backtested.

Week 2: Bandit models

Goal: implement learning models.

Tasks:

- Implement epsilon-greedy.
- Implement UCB.
- Implement Thompson Sampling.
- Implement LinUCB / contextual bandit.

- Add reward function with turnover, downside, and drawdown penalties.
- Compare raw-return reward vs complex reward.

End of week 2 deliverable:

Bandit strategies run in the same backtesting framework as benchmarks.

Week 3: Interpretation and presentation

Goal: make the project convincing.

Tasks:

- Final performance tables.
- Allocation-over-time plots.
- Regime interpretation.
- Transaction cost sensitivity.
- Robustness checks.
- Slide deck.
- Assign speaking parts.

End of week 3 deliverable:

Full presentation with defensible results and clear limitations.

Recommended team split

Person 1: Data and features

Owns:

- Fama-French download
- cleaning
- feature engineering
- train/test split

Person 2: Backtesting and metrics

Owns:

- portfolio return calculation

- turnover
- transaction costs
- Sharpe / drawdown / Sortino / Calmar
- benchmark strategies

Person 3: Bandit models

Owns:

- epsilon-greedy
- UCB
- Thompson Sampling
- LinUCB

Person 4: Interpretation and presentation

Owns:

- regime analysis
- plots
- final story
- slide design
- limitations

Minimum viable version

Do not lose sight of the MVP:

Monthly factor allocation using allocation-rule actions, complex reward, UCB/Thompson Sampling/LinUCB, compared against static equal-weight and rolling Sharpe benchmarks.

That is already strong.

Everything else is optional.